

海星数据权限隔离

🔍 | ⓘ 正文 5552 字 (5.8 页) | 浏览 4 次 (1 人) | 全部 >

 AI 速览

点击生成简要速览，支持更多

风格切换 ▾

助你高效阅读

▼

1. 背景分析

1.1 项目背景

- 需求文档：[【PRD】海星平台数据权限进行隔离](#)
- 众包模式下，服务人员的管理模块趋于稳定，随着开城不断加快，希望对数据进行隔离，避免数据泄露给友商的风险。基于这个现状，对海星平台的数据权限进行隔离。
- 数据隔离产品方案如下：
- 在海马配置城市 - 城市经理的对应关系、海星平台配置培训老师 - 城市 - 网格的对应关系（参考众包模式）
 - 城市经理**：海星系统的订单列表、工单管理、质检管理、网格信息（二级）、服务人员列表、库存查看、招聘管理；城市经理只能查看自己有权限的城市数据，包含筛选项，仅展示有权限的城市，其余页面不做限制
 - 培训老师**：海星系统的服务人员列表、网格信息（二级）、库存查看页面；培训老师只能查看自己有权限的网格数据，包含筛选项，仅展示有权限的数据
 - 数据颗粒度有4种类型：**全量数据、行业内全量数据（和行业交叉）、城市、网格**，没有配置颗粒度的角色无权限访问任何数据；服务商端根据服务商的逻辑走，不做单独的数据隔离（服务商包含加盟商、人资商、薪税商等）

角色大致可以分为：超管、城市经理、培训老师、站长、运营等

本人负责的界面：订单管理、质检管理、判责管理、招聘管理、服务信用管理

- 页面控制由uac做，不需要鉴权（uac 统一权限控制）
 - 相同页面，不同角色可见数据范围不一致，多半是城市权限与网格权限区分，获取城市、网格信息
 - 相同页面，不同角色页面元素不一致，需区分具体权限

1.2 项目目标

1) 业务目标

- 确保在众包模式下，随着业务扩展和城市开拓，数据能够有效隔离，避免数据泄露给竞争对手，保护公司的核心数据资产。

2) 技术目标

- 调整和优化现有的UAC角色权限，删除不必要的权限，新增必要的权限，确保各角色的权限配置符合业务需求。
- 通过建设统一的数据权限管理模块实现数据颗粒度的细化管理，包括全量数据、行业内全量数据、城市、网格、空等类型，确保不同角色的数据访问权限精确到具体颗粒度
- 确保在实现数据隔离和权限管理的过程中，系统的稳定性和扩展性不受影响，能够支持业务的快速扩展和城市开拓。

2. 需求分析（F）

2.1 功能分析

1) 业务用例：

- _____

角色分类	行业	更改前角色名称	更改后角色名称	数据颗粒度	订单管理（运营侧）	订单列表（加盟商侧）	无忧订单（加盟商侧）	工单管理	判责工单列表	质检管理	质检订单列表	网格信息（一级）	网格绘制	网格信息级）
开发优先级					2	不需要数据隔离，有自己的分配逻辑	无需数据隔离		5		6		无需数据隔离	3
服务商	维修	服务商公司负责人-维修（三方服务商）	加盟商公司负责人-维修	随加盟商		√								
	保洁	众包-公司负责人-保洁	薪税商公司负责人-保洁	无										
	保洁	服务商-普通人员-保洁	加盟商公司员工-保洁	随加盟商		√								
	管道疏通	服务商-公司负责人-管道疏通	加盟商公司负责人-管道疏通	随加盟商			√							
	保洁	服务商-公司负责人-保洁	加盟商公司负责人-保洁	随加盟商		√								
城市经理	保洁	运营-城市经理-保洁	城市经理-保洁	城市	√			√	√	√	√	√		√
站长	保洁	运营-城市运营-众包站长	站长-保洁	网格	√			√	√	√	√	√		√
培训老师	保洁	培训老师-保洁		城市	√			√	√	√	√			√
运营	全行业	运营-保险运营	保险运营-全行业-保险运营	无										
	保洁	运营-总部运营-结算运营	总部结算运营-保洁	行业内全量数据	√									
	全行业	运营-总部运营-履约管理	履约管理运营-全行业	全量数据	√			√	√	√	√	√		√

	洗涤	客服-洗涤	判责人员-洗涤	行业内全量数据				√	√					
	洗涤	运营-洗涤		行业内全量数据				√	√					
	管道疏通	客服-管道疏通	判责人员-管道疏通	行业内全量数据	√			√	√					
	保洁	客服-保洁	判责人员-保洁	行业内全量数据	√			√	√	√	√			
	全行业	运营-总部超级管理员-非菜单资源权限	运营-全行业-代客登录权限	无										
	全行业	运营-总部运营-判责	总部判责运营-全行业	全量数据	√			√	√					
	管道疏通	运营-总部运营-管道疏通	总部运营-管道疏通	行业内全量数据	√			√	√					
	保洁	运营-总部运营-保洁	总部运营-保洁	行业内全量数据	√			√	√	√	√	√		√
	全行业	运营-总部超级管理员	产运研-全行业-超级管理员	全量数据	√			√	√	√	√	√	√	√
研发	全行业	研发-保洁自有品牌-前后端	研发-全行业		√			√	√			√		
	全行业	研发-技术功能灰度	研发-全行业-新技术功能试用	全量数据									√	
开换锁项目暂时不动 (业务侧结论：挑城市试点，强要求商家录入服务人员用接单宝权限先不处理)	开换锁	服务商-公司负责人-开锁换锁	加盟商公司负责人-开锁换锁	随加盟商										
		运营-总部运营-开锁换锁												
		运营-城市运营-开锁换锁												
		专家团-开锁换锁												
删除角色		运营-城市运营-管道疏通												
		专家团-管道疏通												
		运营-总部运营-保洁-派单同学										√		
		判责人员						√	√					
		三方加盟商公司负责人-保洁			√									



	三方加盟商公司运营-保洁			√									
	研发-无忧X-前后端						√	√					

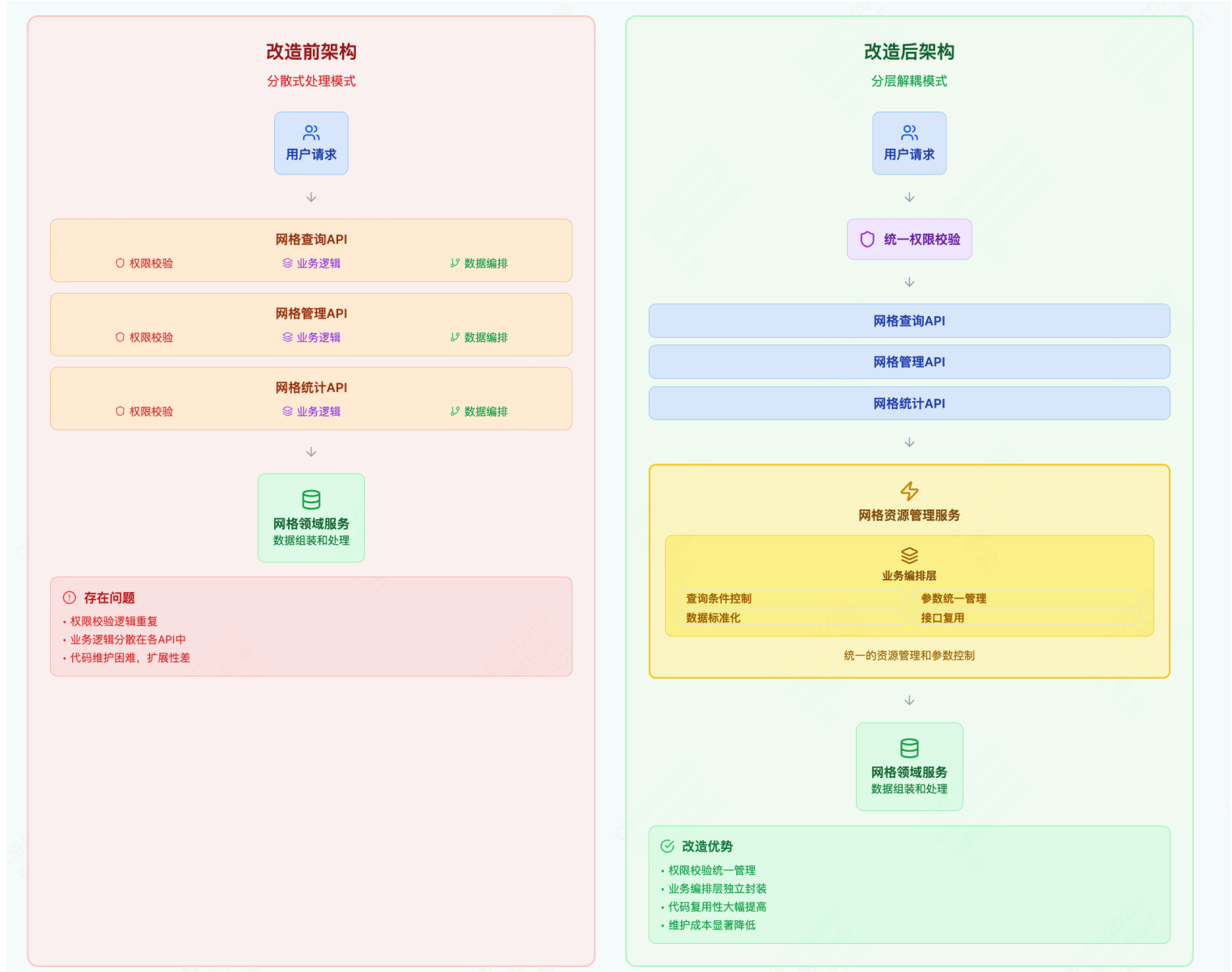


3. 系统设计（A）

3.1 核心流程设计

3.1.1 整体流程

i 通过搭建资源管理服务，将上游分散的数据查询逻辑以及权限相关逻辑进行收归，以网格相关的资源举例，改造前后差异如下图所示：



3.2 核心模型设计

i 新增培训人员-网格关联关系表

3.3 关键接口定义

- 【强制】 查询接口设计需要明确定义返回字段选项，约束返回全量信息的使用场景（甚至拒绝返回）；

【强制】 批量接口设计需要明确定义对请求/响应数据的数量约束；

【强制】 写入接入需要对接口是否保证幂等进行说明（幂等接口需明确幂等 Key 逻辑）；

关键接口约束说明（[建议直接粘贴服务接口文档链接](#)）

接口方法定义	接口功能说明	调用模式	入参校验约束	返回结果说明	幂等性说明	并发风险评估	SLA 预估
		同步/异步	【必填】 批量数量限制： 【必填】 返回模块选项： 【可选】 xxx	返回数量限制：			

工具类

测试环境测试账号：

- 众包站长：liumu

城市经理：xudingkun - 上海市 [海马城市配置](#)

超管：xiezychao

培训老师：zhangcongliang -上海市 [海马城市配置](#)

^ maven

XML

```
1 <dependency>
2     <groupId>com.sankuai.lifevent</groupId>
3     <artifactId>sp-http-common-utils</artifactId>
4     <version>1.1.3-jtc-SNAPSHOT</version>
5 </dependency>
6 <dependency>
7     <groupId>com.sankuai.lifevent</groupId>
8     <artifactId>sp-common</artifactId>
9     <version>1.3.5-jtc-SNAPSHOT</version>
10 </dependency>
```

开发示例：

1、根据用户配置城市展示内容

^ 代码块

Java

```
1 接口： /api/faas/sp/schedule/classes/add
2 描述： 新增班次
```



```
3  要求：城市经理限制城市
4
5  // 注入工具类
6  @Resource
7  private UserAuthHelper userAuthHelper;
8
9  @Override
10 public ApiResponse<Boolean> execute(AddClassReq req, FbiApiContext fbiApiContext) {
11     if (!validateParams(req)) {
12         return new ApiResponse<>(SpReturnCodeEnum.H_PARAM_ERROR.getCode(), "参数错误");
13     }
14     log.info("AddClassHandler req:{})", JSON.toJSONString(req));
15     try {
16         // 数据可见范围
17         CityAccessScope cityAccessScope = userAuthHelper.loadCityAccessScope(fbiApiContext);
18         // 城市权限校验
19         cityAuthCheck(req.getCityIds(), cityAccessScope);
20         // 其他逻辑
21         others();
22         return new ApiResponse<>(SpReturnCodeEnum.H_SUCCESS.getCode(), SpReturnCodeEnum.H_SUCCESS.getMsg(), true);
23     } catch (SpAuthException e) {
24         log.warn("AddClassHandler error! req:{})", JSON.toJSONString(req), e);
25         return new ApiResponse<>(e.getCode(), e.getToastMsg());
26     } catch (Exception e) {
27         log.error("AddClassHandler error! req:{})", JSON.toJSONString(req), e);
28         return new ApiResponse<>(SpReturnCodeEnum.H_FAIL.getCode(), SpReturnCodeEnum.H_FAIL.getMsg());
29     }
30 }
31 // 城市权限校验
32 private void cityAuthCheck(List<Integer> cityIds, CityAccessScope cityAccessScope) {
33     if (cityAccessScope.isAllVisible()){
34         return;
35     }
36     for (Integer cityId : cityIds) {
37         if (!cityAccessScope.getVisibleCityIds().contains(cityId)){
38             throw new SpBizException(SpReturnCodeEnum.H_NO_AUTH.getCode(), "未配置该城市权限");
39         }
40     }
41 }
```

2、根据用户角色展示内容

代码块

Java

1 接口：/api/faas/sp/technician/show

2 描述：服务人员列表页前端渲染

3 要求：限制城市经理/培训老师可见数据范围

4

5 // 注入工具类

6 @Resource

7 private UserAuthHelper userAuthHelper;

8

9 @Override

10 public ApiResponse<TechnicianShowDTO> execute(Integer bizType, FbiApiContext fbiApiContext) {

11 try {

12 // 用户权限

13 UserAuthModel userAuthModel = userAuthHelper.loadUserAuthModelByCondition(fbiApiContext,

14 Lists.newArrayList(UserAuthFieldEnum.FAKE_LOGIN_INFO, UserAuthFieldEnum.UAC_TAG, UserAuthFieldEnum.USER_INFO));

15 CityAccessScope cityAccessScope = userAuthHelper.loadUserAuthModelByCondition(userAuthModel.getIamRoleType());

16

17 // 根据角色获取不同数据组装

18

19 // 过滤不可见城市数据

20

21 return ApiResponse.success(XXX);

22 } catch (SpAuthException e) {

23 log.warn("XXX error! req:{}, JSON.toJSONString(req), e);

24 return new ApiResponse<>(e.getCode(), e.getToastMsg());

25 }

26 }

数据可见范围：

运营/超管：全部人

城市经理/培训老师：配置城市 ：服务人员所在城市

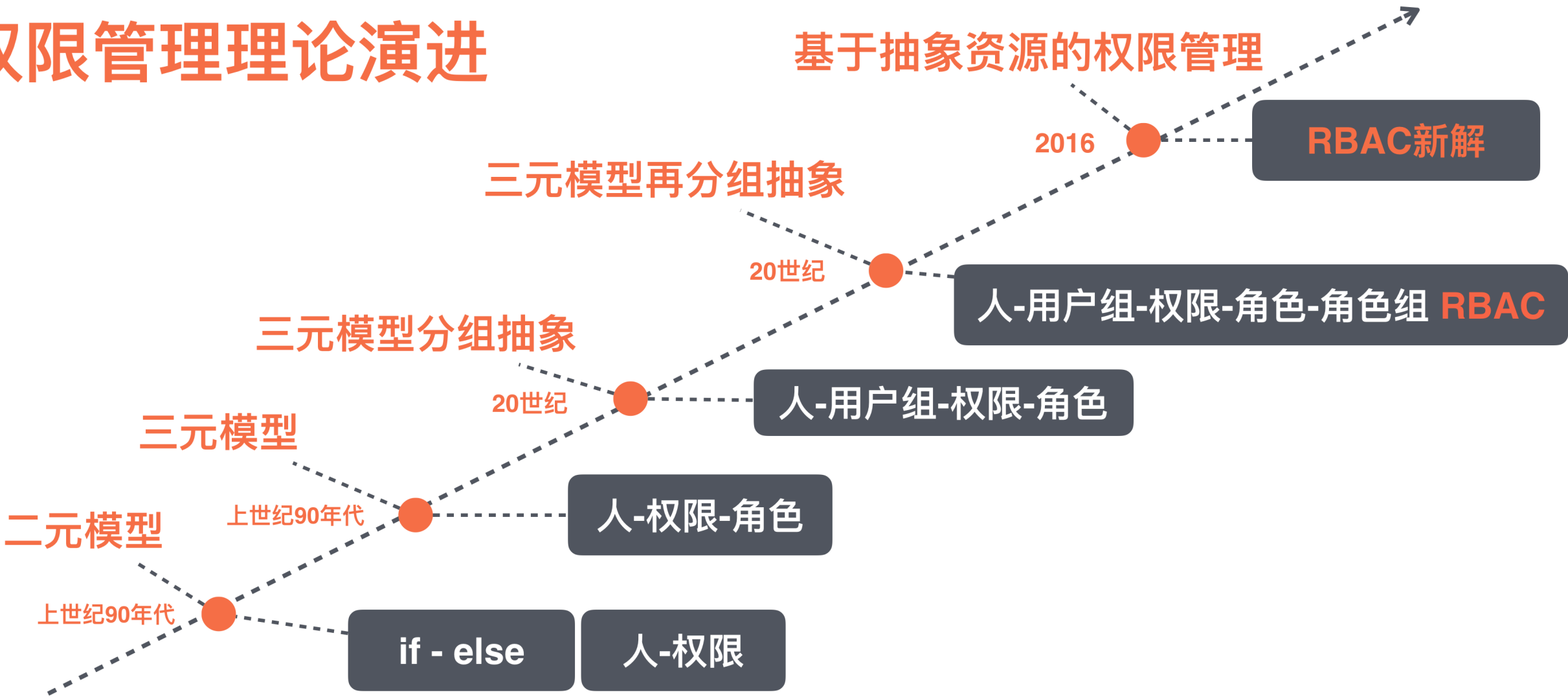
众包站长：未配置布点+配置布点网格内 ：

服务商：服务商下的人

统一权限管理系统

权限管理模型

权限管理理论演进



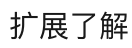
上世纪90年代开始提出权限管理的**二元模型**，通过用简单的 if-else 的硬编码形式，来设置某人在什么条件下是否可以执行某操作。随着系统的复杂性提升，使用场景的增加，用户增多，对权限集合的划分就有了 交集，并集，分立元素的概念，[角色] 的概念就需要从 [权限] 的概念中剥离出来，从而诞生出现了**三元模型**。

三元模型解决了大多数场景下的权限管理需求，但也是非常脆弱的。一个极小的权限变动都可能导致大量的调整，其中还常常伴随大量权限集合的拆分与组合，于是诞生了 [用户组] 的概念。在许多老系统(Windows97)中依然能看到**三元分组模型**的身影。

再后来，角色上又加了一层角色组，成为了称霸整个20甚至21世纪的权限管理模型：**RBAC模型**。NIST（The National Institute of Standards and Technology，美国国家标准与技术研究院）标准 RBAC模型由4个部件模型组成，这4个部件模型分别是基本模型RBAC0（Core RBAC）、角色分级模型RBAC1（Hierarchal RBAC）、角色限制模型RBAC2（Constraint RBAC）和统一模型RBAC3（Combines RBAC）。

RBAC权限模型

- **RBAC**：基于角色的权限访问控制（Roles Based Access Control）。RBAC支持三个著名的安全原则：最小权限原则，责任分离原则和数据抽象原则。
- **统一权限管理**：指根据系统设置的安全规则或安全策略，用户可以访问而且只能访问自己被授权的跨系统资源。



- RBAC概念讲解
- RBAC0/1/2/3详解

统一权限管理UAC

UAC概述

1. 随着美团点评体量的扩张，员工数量的上升，身份的划分（如员工，供应商，合作伙伴以及客户），公司主体的增长(控股公司/生态公司)，有效地对权限进行分配成为一个重大挑战。另一方面，企业应用，服务以及API调用量不断增长，使得涉及权限的管理成为另一个重大挑战。UAC统一权限管理应对这一系列挑战，承担起美团点评内部应用及跨企业应用，服务和API权限管理的复杂任务。
2. UAC采用业界最广泛使用的**RBAC模型**并进行扩展，提供了一套**平台无关**的权限管理模型，使企业架构师能够在现有的业务上实施统一的安全管理层。
3. UAC基于RBAC3模型，在权限层级下设计了一套资源模型。租户-用户(动态组)-角色-权限-资源(页面)-资源(前端模板)-资源(API)，通过元数据的方式，支持各种类型的权限表示形式。

UAC领域模型

-
1. UAC：基于通用的RBAC模型，为美团点评内部业务系统提供符合实际业务的功能/数据权限管理并独立于业务层的**统一权限管理通用服务**，搭建完整的**权限审计机制**。
 2. UAC的业务方：美团点评内部**各业务系统**。
 3. UAC的使用方：各业务系统的PM\RD等**研发人员、业务运营人员、权限审计人员**。
 4. UAC最终目标：有效控制终端用户使用各业务系统资源的**权限**。

UAC解决方案

- **权限控制**：UAC基于元数据的设计，几乎为所有权限的控制模式提供了支持的能力，目前常用的鉴权逻辑有以下几种：

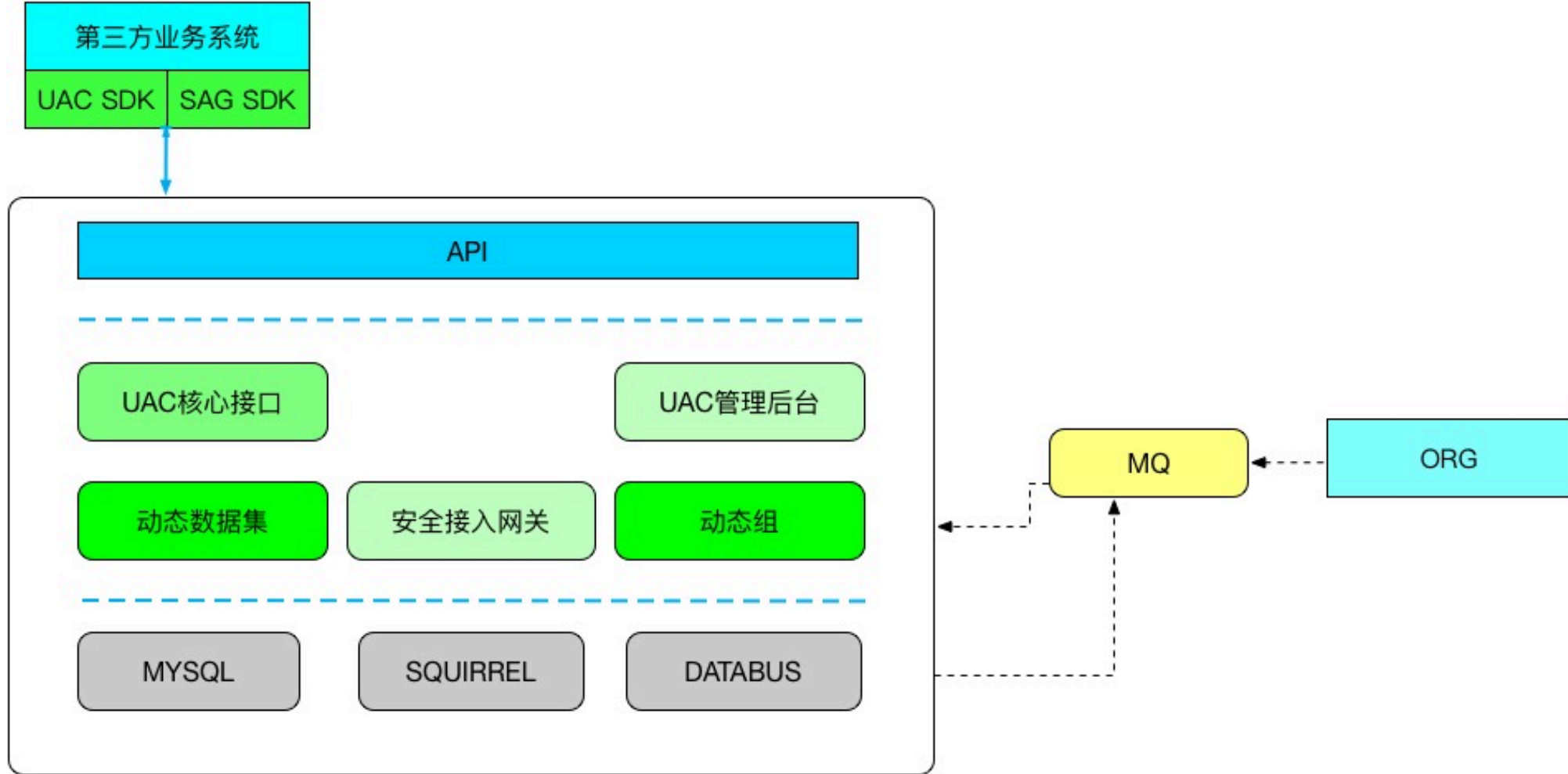
	支持方式	支持程度
URL鉴权	Shiro Filter / Servlet Filter	准Rest 接口控制
API鉴权	Shiro Filter / Servlet Filter / Thrift Interceptor	系统间 Rest/Thrift 调用控制
前端模板控制	前端模板	根据权限渲染不同的前端界面
菜单树控制	Api接口	根据权限生成页面的树状结构

- **权限管理**：UAC开发的过程中，花费约70%的工作量，处理权限的分配、管理问题。目前常用的权限分配有以下几种：

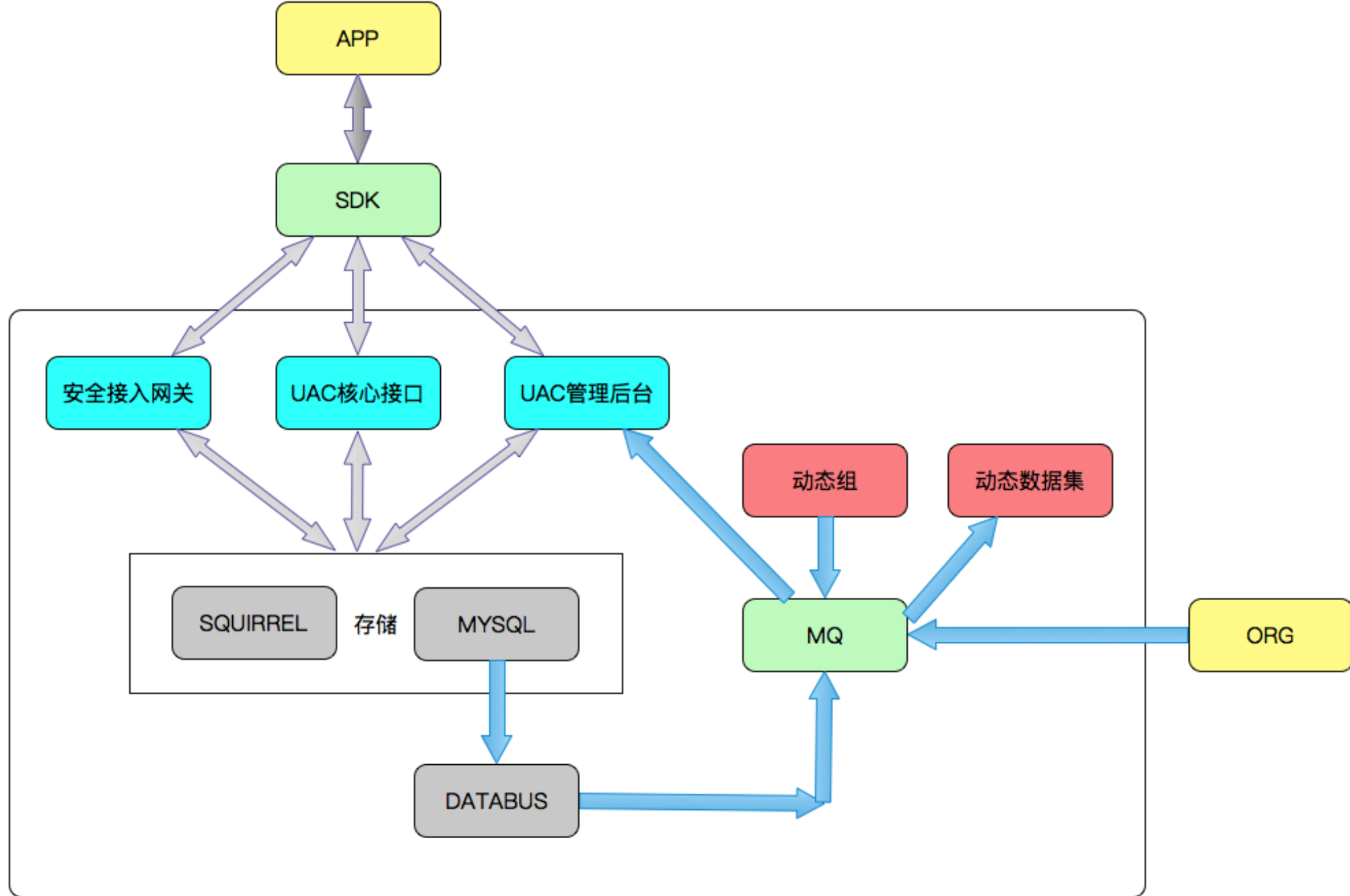
	配置方式	说明
手动授权	UAC管理后台	手动授予某人某角色，或进行摘除
动态组授权	UAC管理后台	通过 人/部门/职级/岗位/员工类型/岗位 等数十种条件自由组合，根据员工入转调离的动作，自动进行授权
API授权	SDK	通过API进行角色的授予，摘除

目前UAC已经实现了**权限自我管理**，形成了权限管理链路上的闭环。

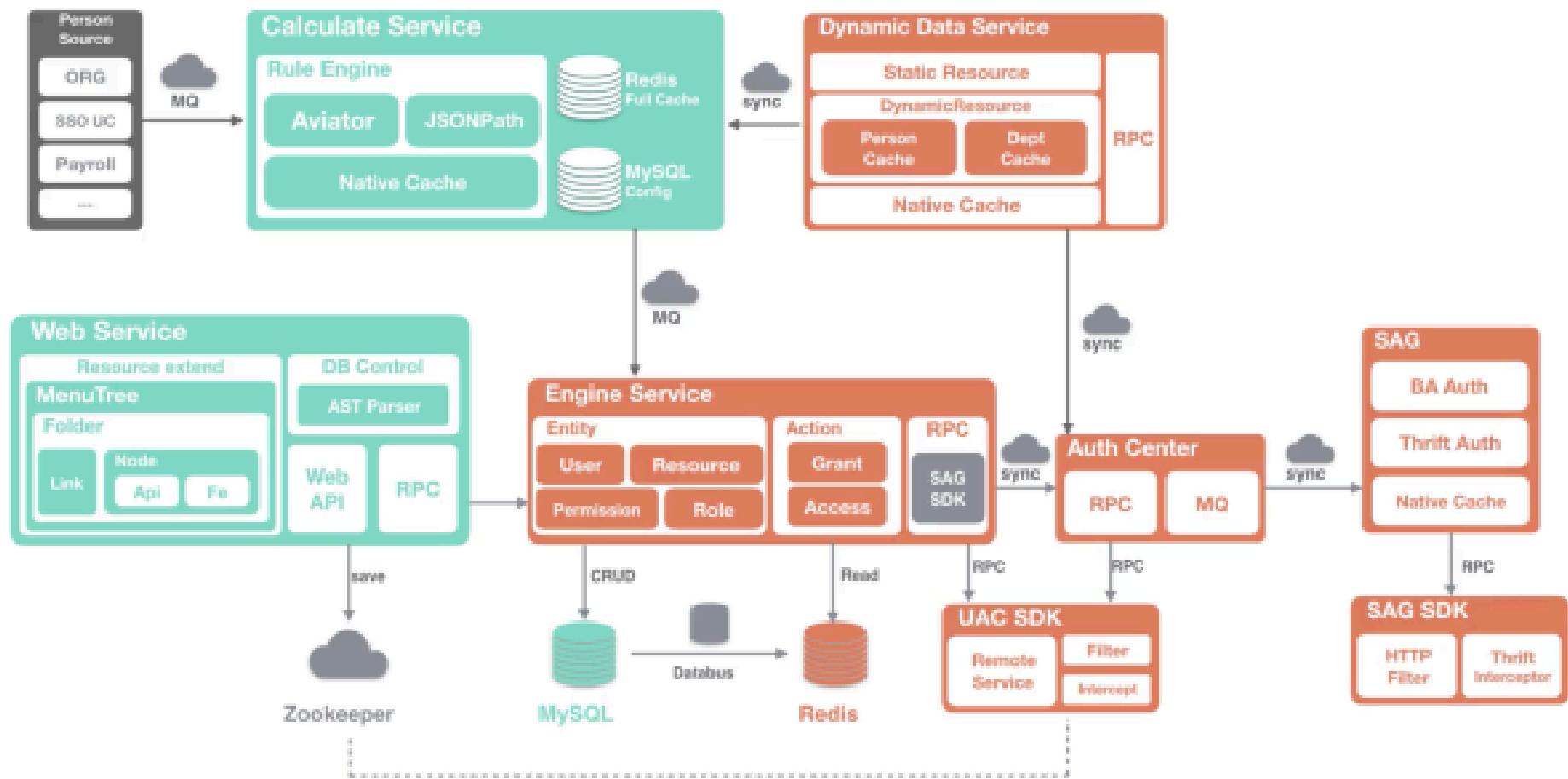
UAC系统架构



系统结构图

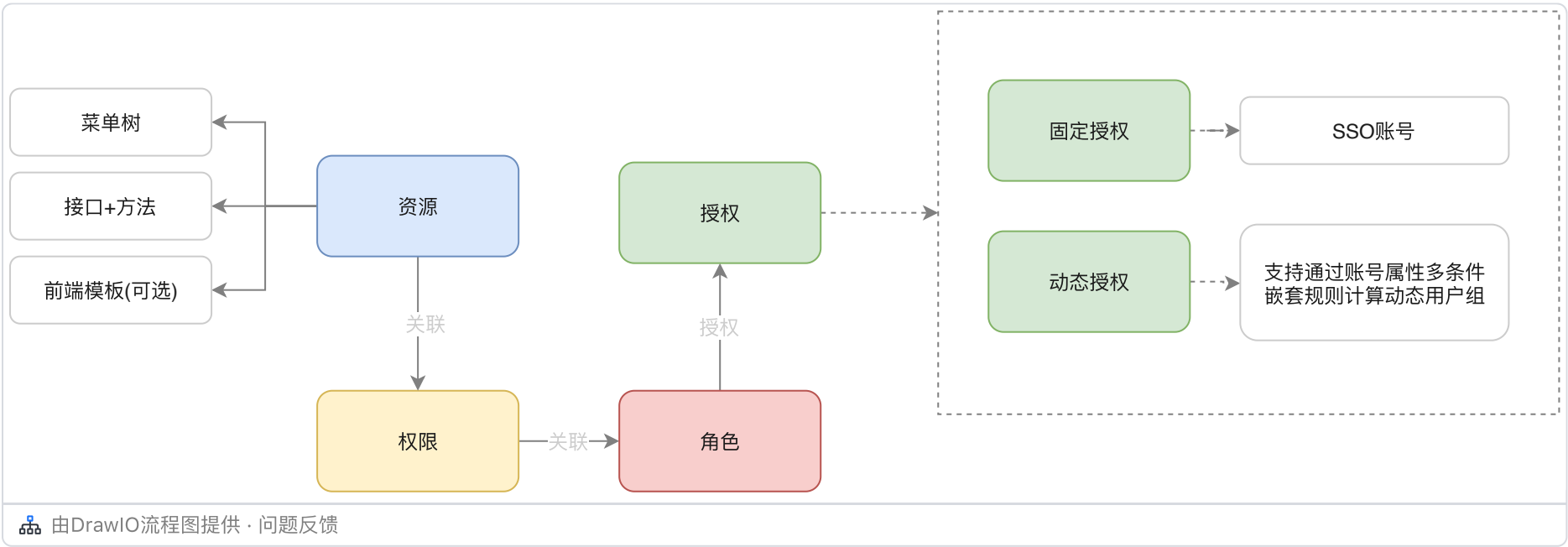


系统调用关系图

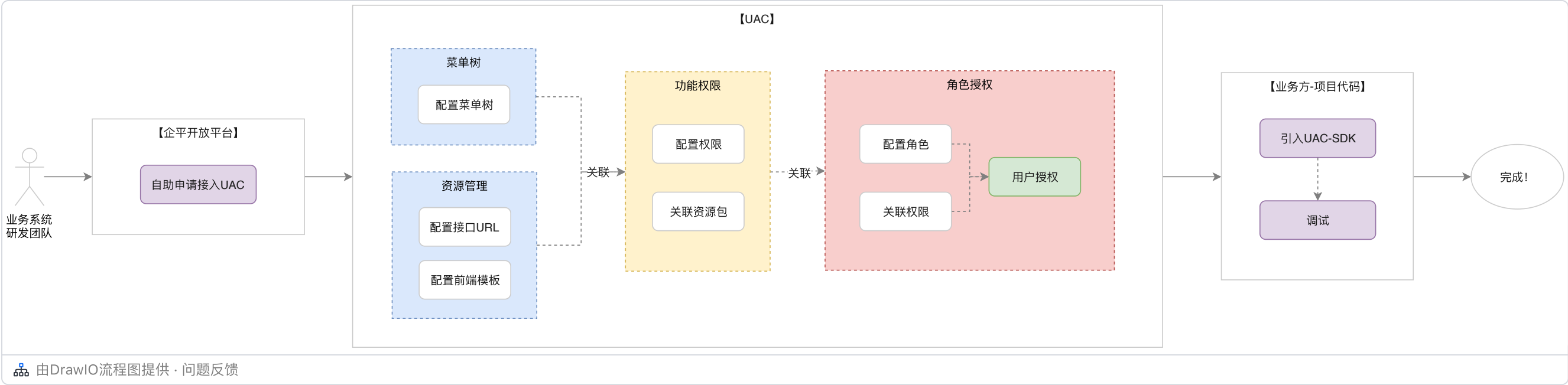


授权模型+流程

授权模型



在UAC中，菜单、API、前端模板都属于系统资源。系统资源通过权限-角色进行打包，最终把用户和角色关联上，即完成授权。



接入指南

第1步：点击进入企平开放平台【测试环境：<http://openplatform.it.test.sankuai.com/>，线上环境：<https://open.sankuai.com/>】注册应用（从线上注册，会自动同步线下），完成应用认证审批。

第2步：选择接入UAC资源，完成审批后，点击进入UAC权限系统【测试环境：<http://uac.it.test.sankuai.com>，线上环境：<http://uac.vip.sankuai.com>】进行权限配置。

第3步：在企平开放平台/UAC可获取业务系统的APPKEY和secret，且仅接入UAC的申请人自动同步为UAC研发管理员和权限管理员。

前面三步是前置操作，获取appkey和secret，下面java项目介入UAC

1、引入UAC-SDK

代码块XML

```
1 <dependency>
2   <groupId>com.sankuai.meituan</groupId>
3   <artifactId>uac-common-sdk</artifactId>
4   <version>1.0.32</version>
5 </dependency>
```

2、配置UAC Filter

由于UAC SDK 使用了SSO SDK 的这个方法（User user = UserUtils.getUser()）来获取当前登录用户，因此接入UAC之前，需要先接入SSO。

springXML

```
1 <bean id="uacAuthFilter" class="com.sankuai.meituan.uac.sdk.filter.UacFilterFactoryBean">
2   <!--必须配置，以下二者需先到开放平台(http://open.sankuai.com) 申请接入UAC后颁发，对应企平开放平台的AppKey和AppSecret -->
3   <property name="appKey" value="替换成开放平台颁发的AppKey"/>
4   <property name="secret" value="替换成相应的AppSecret"/>
5
6   <!--接入的UAC HOST地址，线下为: http://uac.it.test.sankuai.com 线上为: http://uac.vip.sankuai.com -->
7   <property name="host" value="http://uac.it.test.sankuai.com"/>
8
9   <!-- 以下配置为可选配置 -->
```

```
10
11 <!-- 表示需要经过UAC鉴权的URI，多个之间以','分割,支持Ant风格路径表达式。includedUriList中的URI必须经过SSO进行过滤，否则UAC无法获取当前登录人信息进行鉴权。 -->
12 <property name="includedUriList" value="/test/**,/api/**" />
13
14 <!-- 表示不需要经过UAC鉴权的URI。一定要看的注意：两者都配置的情况下，excludedUriList会失效，includedUriList优先级更高。 -->
15 <property name="excludedUriList" value="/octo/checkAlive/**,/pass/**"/>
16
17 <!-- UAC授权失败后返回的错误信息 -->
18 <property name="authFailedResponse" value="业务方自行设置成期望的错误信息格式，可以参考配置项说明！"/>
19
20 <!-- 是否关闭UAC日志功能（URI鉴权过程打印一些日志，方便调试），true表示关闭，默认为false，即开启日志功能。 -->
21 <!-- <property name="logClosed" value="true"/> -->
22
23 <!-- URL鉴权时是否需要对HTTP方法进行鉴权 -->
24 <!-- <property name="authUrlWithMethod" value="true"/> -->
25 </bean>
```

^ springboot

Java 

```
1 package com.example.demo;
2
3 import com.sankuai.meituan.uac.sdk.filter.UacFilterFactoryBean;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.boot.web.servlet.FilterRegistrationBean;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
8 import org.springframework.web.filter.DelegatingFilterProxy;
9
10 import static javax.servlet.DispatcherType.REQUEST;
11
12 @Configuration
13 public class UacFilterConfiguration {
14     @Bean
15     public FilterRegistrationBean uacFilter() {
16         DelegatingFilterProxy filter = new DelegatingFilterProxy();
17         FilterRegistrationBean registration = new FilterRegistrationBean();
18         filter.setTargetBeanName("uacFilterBean");
19         filter.setTargetFilterLifecycle(true);
20
21         registration.setFilter(filter);
22         registration.addUrlPatterns("/*");
23         registration.setDispatcherTypes(REQUEST);
24         registration.setName("uacFilter");
25
26         // 注意顺序应设置在SSO filter之后
27         registration.setOrder(2);
```

```
28         return registration;
29     }
30
31     /**
32      * uacFilterBean配置
33      */
34     @Bean
35     public UacFilterFactoryBean uacFilterBean() {
36         UacFilterFactoryBean filterFactoryBean = new UacFilterFactoryBean();
37         // 必须配置，以下二者需先到开放平台(http://open.sankuai.com) 申请接入UAC后颁发，对应企平开放平台的AppKey和AppSecret
38         filterFactoryBean.setAppKey("替换成从Lion/KMS获取");
39         filterFactoryBean.setSecret("替换成从Lion/KMS获取");
40
41         // 接入的UAC HOST地址，线下为: http://uac.it.test.sankuai.com 线上为: http://uac.vip.sankuai.com
42         filterFactoryBean.setHost("http://uac.it.test.sankuai.com");
43
44         // -----以下配置为可选配置-----
45         // 表示需要经过UAC鉴权的URI，多个之间以','分割, 支持Ant风格路径表达式。
46         // includedUriList中的URI必须经过SSO进行过滤，否则UAC无法获取当前登录人信息进行鉴权。
47         filterFactoryBean.setIncludedUriList("/test/**,/api/**");
48
49         // 表示不需要经过UAC鉴权的URI。两者都配置的情况下，excludedUriList会失效，includedUriList优先级更高。
50         filterFactoryBean.setExcludedUriList("/octo/checkAlive/**,/pass/**");
51
52         // UAC授权失败后返回的错误信息
53         filterFactoryBean.setAuthFailedResponse("{\"status\":0,\"message\":\"您没有权限访问\", \"version\":\"1.0.0-SNAPSHOT\"\", \"data\":{\"message\":\"您没有权限访问\"}}");
54
55
56         // 是否关闭UAC日志功能（URI鉴权过程打印一些日志，方便调试），true表示关闭，默认为false，即开启日志功能。
57         //filterFactoryBean.setLogClosed(true);
58
59         // URL鉴权时是否需要对HTTP方法进行鉴权
60         // filterFactoryBean.setAuthUrlWithMethod(true);
61         return filterFactoryBean;
62     }
63 }
```

配置项	含义	是否必须	默认值
appKey	应用开发中心申请接入UAC成功后颁发的AppKey	是	无
secret	应用开发中心申请接入UAC成功后颁发的AppSecret	是	无
host	需要接入的UAC HOST地址	是	线下： http://uac.it.test.sankuai.com 线上： http://uac.vip.sankuai.com



includedUriList	表示需要经过UAC鉴权的URI，多个之间以','分割,支持Ant风格路径表达式。includedUriList中的uri必须经过SSO进行过滤，否则UAC无法获取当前登录人信息进行鉴权。	否	二者选其一配置， 如果都配置，仅includedUriList生效 如果都不配置，值为/**
excludedUriList	表示不需要经过UAC鉴权的URI	否	
authFailedRedirectUrl	UAC授权失败后重定向地址。若未配置，则返回authFailedResponse配置项指定的JSON格式错误信息。	否	无默认值 注：该配置项为1.0.10版本新增配置项
authFailedResponse	UAC授权失败后返回的错误信息	否	<pre>{ "status": 0, "message": "您没有权限访问", "version": "1.0.0-SNAPSHOT", "data": { "message": "您没有权限访问" } }</pre>
logClosed	是否关闭UAC日志功能（URI鉴权过程打印一些日志，方便调试），true表示关闭，默认为false，即开启日志功能。	否	false
authUrlWithMethod	URL鉴权时是否需要为HTTP方法进行鉴权	否	false